Week 5 · Checking input

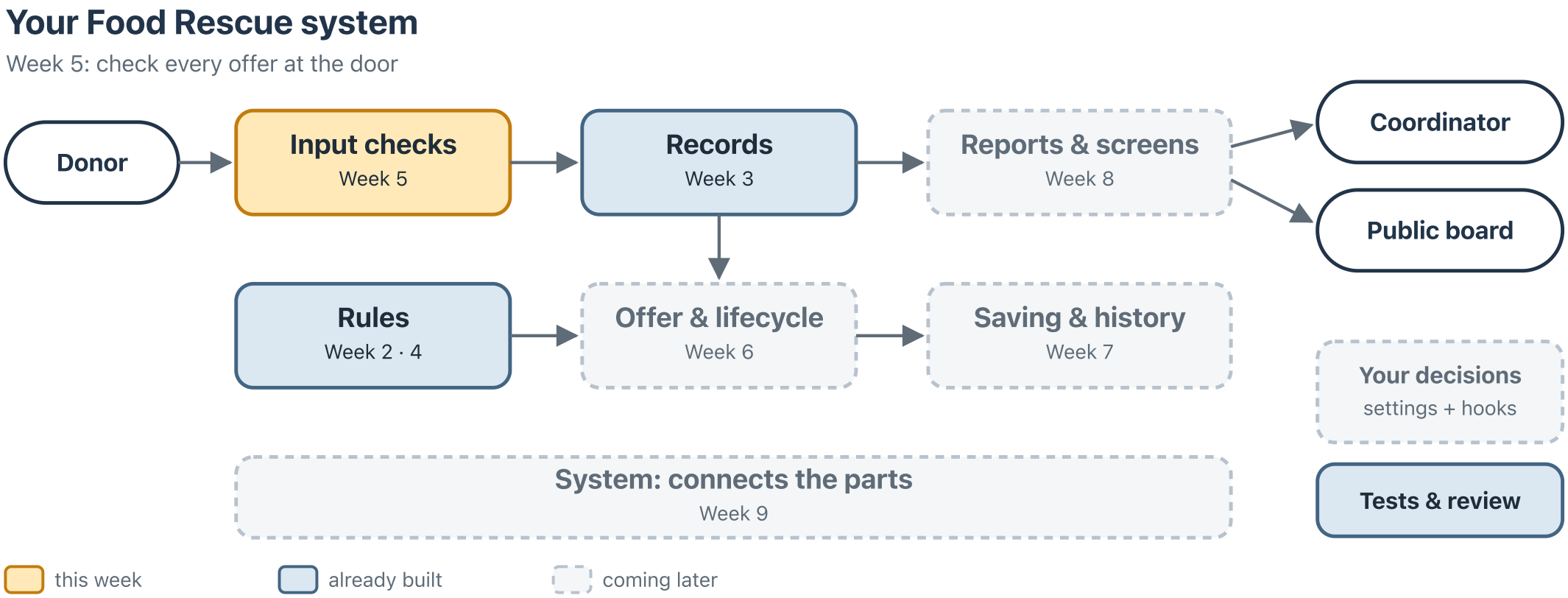
Programming for Information Systems

Never trust input: check at the door

This week

Question	What should a system do when the input is wrong?
Python	`strip`, `lower`, `int()`, `try` / `except ValueError`, a list of errors, two return values
Principle 5	Never trust input
You build	`clean_offer(raw)` → `(offer, errors)`

Your system so far



The idea: check at the door

- At the airport, staff check your passport **before** you board.
- A system has doors too: a form, a file, a message.
- The door is the **cheapest** place to stop bad data.
- Once bad data is inside, it spreads into totals, reports and decisions.

> People are not careless on purpose. A good system **expects** wrong input.

Every value from a file is text

```
donor,contact,food,category,portions,hours_left
Sunrise Bakery,sunrise@example.org,Bread rolls,bread,30,5
Student Club Bake Sale,club@example.org,Cupcakes,snacks,many,6
Hotel Kitchen,hotel@example.org,Steamed rice,rice meals,1200,8
,,Mango slices,fruit,10,6
```

```
rows = fr.validation.read_offers_csv("food_rescue_kit/data/offers.csv")
print(rows[0]["portions"], type(rows[0]["portions"]))   # 30 <class 'str'>
```

- A CSV file is only characters. It cannot say "this is a number".

Tidy text: `strip()` and `lower()`

```
typed = "  Dara "
print(typed == "Dara")                  # False
print(typed.strip() == "Dara")          # True
print(" Bread ".strip().lower())        # bread
print(" Bread " in CATEGORIES)          # False
print(" Bread ".strip().lower() in CATEGORIES)   # True
```

- A **method** is a function that belongs to a value: `text.strip()`.
- It returns **new** text; the original is unchanged.

`int()`: text to a whole number, or an error

```
print(int("30") + 4)      # 34
print(int(" 12 "))        # 12   (spaces at the ends are fine)
print(int("-3"))          # -3   a whole number, but sensible?
print(int("many"))        # ValueError
```

```
ValueError: invalid literal for int() with base 10: 'many'
```

- Converting is one check. The **range** is a second check.

`try` / `except ValueError`

```
quantity_text = "two"
try:
    quantity = int(quantity_text)
    print("Order for", quantity, "phones")
except ValueError:
    print("Please type the quantity as a number, for example 2.")
```

- Keep `try` **small**: only around the conversion.
- Catch **only** `ValueError`, the problem you expect.

Collect every problem

```
errors = []
if name.strip() == "":
    errors.append("name is required.")
try:
    age = int(age_text)
except ValueError:
    errors.append("age must be a whole number, for example 19.")
```

- Check **everything**, so the person can fix all problems at once.
- Plain words: **what** is wrong and **how** to fix it.

Return two values, unpack them

```
def check_age(age_text):
    try:
        age = int(age_text)
    except ValueError:
        return None, ["age must be a whole number, for example 19."]
    if age < 16:
        return None, ["volunteers must be 16 or older."]
    return age, []

age, errors = check_age("17")     # 17 []
```

Principle 5 · Never trust input

1. **Check at the door**, before anything is saved.
2. **Explain** every problem in plain words.
3. **Change nothing** when the input is bad.

> Trust is not about people. Honest people still type "many".

⚖️ Good or risky?

Version A (from an AI assistant)

```
def add_offer(raw):
    offer = {
        "donor": raw["donor"],
        "category": raw["category"],
        "portions": int(raw["portions"]),
    }
    saved_offers.append(offer)
    return "OK"
```

It crashed on "many". You said so.

Version B ("Fixed! It will never crash.")

```
def add_offer(raw):
    offer = {"donor": raw["donor"]}
    saved_offers.append(offer)  # save first
    try:
        offer["portions"] = int(raw["portions"])
    except:
        offer["portions"] = 0   # never crash
    if raw["category"] not in CATEGORIES:
        return "error"
    offer["category"] = raw["category"]
    return "OK"
```

What Version B did to the sample day

Row	What B saved	Who is harmed
Cupcakes, "many"	0 portions	the Student Club thinks it was accepted; nobody collects
Noodle soup, "soup"	half an offer, no category	the coordinator gets a broken record
Steamed rice, 1200	accepted	a team goes for food no partner can take
Mango slices, no donor	accepted	nobody knows where to collect

Silently wrong is worse than a clear refusal. A crash is noticed; a quiet 0 goes into the totals.

You build: `clean_offer(raw)`

Input	Return
a good row	`(offer, [])`: text stripped, category lowercase, numbers as `int`
a bad row	`(None, [every problem in plain words])`

- The **donor** reads the messages; the **coordinator** gets clean records.
- The limits are constants: `MIN_PORTIONS`, `MAX_PORTIONS`, `REJECT_IF_HOURS_LEFT_BELOW`, `CATEGORIES`, `REQUIRED_FIELDS`.

Four TODOs

```
def to_whole_number(text):
    ...                          # TODO 1: try int(text), except ValueError → None

def clean_offer(raw):
    donor = raw.get("donor", "")          # TODO 2: .strip() (and .lower())
    ...                                   # required fields: done for you
    if portions is None:
        errors.append("portions must be a whole number, for example 12.")
    # TODO 3: elif portions is outside MIN_PORTIONS..MAX_PORTIONS
    # TODO 4: category must be in CATEGORIES
```

- Most of the function is written for you. You fill in only the key lines.
- Test it on a messy row and a bad row **before** you run the check.

Check it, then plug it in

```
fr.check("clean_offer", clean_offer, MY_SETTINGS)

my_system = fr.System(MY_SETTINGS, hooks={"clean_offer": clean_offer})
my_system.run_day()           # 🚫 = a row YOUR door refused
```

- A door that refuses everything passes most "bad row" checks.
- That is why the **first** case is a good row with extra spaces.
- A refusal must change nothing: `records()` before == after.

🧭 Your decision · what does the door accept?

```
MIN_PORTIONS = 1
MAX_PORTIONS = 200
REJECT_IF_HOURS_LEFT_BELOW = 1
REQUIRED_FIELDS = ["donor", "food", "category", "portions", "hours_left"]
```

- **Portions:** is 1 portion worth a trip? Can any partner take 200?
- **Hours left:** with 1 hour left, can a volunteer still get there?
- **Required fields:** should `contact` be required?
  - **For:** the team can arrange the pickup and call about problems.
  - **Against:** the system holds more personal data, and must protect it.

> Decision log: my choice · an alternative · why · who is affected

This week's tasks

- ⭐ **Core:** P1–P4, the ⚖️ discussion, `clean_offer` with all ✅, plugged in, decision log.
- ⭐⭐ **Stretch:** surprising numbers (`int("9𝟡")` is 12!), one friendly message for the donor.
- 🔎 **Research 4 (5%):** how a real system keeps data and history: what is kept, which IDs, what happens on a correction, who needs the history. Share it in the Week 6 warm-up.

Next week · Objects and lifecycles

A clean offer still has a journey: **new** → **approved** → **picked up** → **delivered**. You will make sure it can only move the right way.